CSM GLEAMING

COMPUTER SCIENCE MENTORS 61A

November 2, 2026 – November 6, 2026

1 Gleam basics

Gleam is a functional programming language, a paradigm where programs are built by applying and composing functions, and function definitions are trees of expressions mapping values to other values rather than sequences of statements that update program state.

Gleam is also statically-typed and type-safe. Type safety is the extent to which a language prevents type errors, such as adding a string to an integer. Type enforcement can be static (at compile time), dynamic (at run-time), or both. In static type systems, type safety usually guarantees that an expression's eventual value belongs to its static type.

When writing functions that operate on a given input, Gleam requires that you handle any theoretically possible input case, even if you've potentially handled such a case earlier. For instance, the function `list.max` (which behaves very similarly to Python's `max` function) does not return an integer directly, but a `Result`. This covers an empty list being passed into `max`: it returns `Ok(value)` or `Error(Nil)` even if you've already ruled out the empty case, guaranteeing an `Ok` result.

Similarly, a `case` statement in Gleam must handle every theoretically possible case, even if that case may not be possible in the context of your own code. For instance, say you have a function that exclusively outputs even integers, and you use one of those integers `n` in a `case` statement:

```
case n % 2 {
    0 -> //some code for the case in which n is even
    _ -> //some code for the case in which n is odd
}
```

Gleam will require you to include the catch-all _ case, regardless of whether `n` could be odd. Even writing a `1` case is not enough: Gleam cannot prove that `n % 2` is only ever `0` or `1` (it is `-1` for negative odd `n`), so an `Int` case always needs _.

1. Implement `factorial`!

```
pub fn factorial(n: Int) -> Int {
  case n > 0 {
    True -> _______ * ____________________________________
    False -> _______
  }
}
```

In regular recursion, our very first function will open a new frame, and each new frame will open new frames until we reach our base case. Typically, every frame must wait for subsequent frames to finish evaluating and return before they can return. That means that right before the base case, we have a lot of calls awaiting the call underneath it before doing something. Because each frame takes up physical space on our computer, recursion can be space inefficient. One way of solving this is tail recursion. If we know that we can write our recursive function in such a way that a previous frame doesn't need to do anything with the later frames, we can close previous frames once we're done with them- saving space on our computer. Here's an example:

```
pub fn factorial(x: Int) -> Int {
  factorial_helper(x, 1)
}

fn factorial_helper(x: Int, curr: Int) -> Int {
  case x {
    0 | 1 -> curr
    _ -> factorial_helper(x - 1, curr * x) // The last thing this function
        does is call itself
  }
  //credit to the Gleam language website for this example
}
```

We've defined this helper function such that it checks our base condition and then immediately returns a recursive call. Because it doesn't do anything else with the result of the recursive call, we have no reason to keep the original frame open, and it will close the first frame to save computer memory. Ultimately, the very last recursive call will be returned with our correct value. Tail recursion only works when a language is tail-call optimized, meaning that the language knows to close unnecessary recursive frames. Python isn't tail-call optimized, so even if you write a tail recursive function, it won't be any more memory efficient. However, Gleam is tail-call optimized, so we can take advantage of this!

2. Implement `sum`, a tail recursive function that returns the sum of all elements of a list of integers.

```
fn sum_helper(l: List(Int), curr: Int) -> Int {
  case l {
    [] -> _______
    [first] -> _______ + _______
    [first, ..rest] -> ___________________________________
  }
}

pub fn sum(l: List(Int)) -> Int {
  sum_helper(_______, _______)
}
```

Gleam supports generics, which let you write functions that work with any type. Instead of naming a concrete type, you use a type variable, written with a lowercase name, and it stands in for whatever type the function is called with. Here are some examples:

```
pub fn main() {
  let add_one = fn(x) { x + 1 }
  let exclaim = fn(x) { x <> "!" }
  echo twice(10, add_one) // Outputs 12. The type variable is Int.
  echo twice("Hello", exclaim) // Outputs "Hello!!". The type variable is
      String.
}

// The name `value` refers to the same type multiple times
fn twice(argument: value, my_function: fn(value) -> value) -> value {
  my_function(my_function(argument))
}
//credit to the Gleam language website for this example
```

We also don't have to restrict ourselves to only doing this with functions! We can do this in type definitions as well (see the Tree type in Gleam).

3. Implement `reduce`, a tail recursive function that applies a two-argument function `f` on the elements of a list from left to right, starting with `start`.

```
fn reduce(l: List(a), f: fn(a, a) -> a, start: a) -> a {
  case l {
    [] -> _______
    [first] -> ___________________________________
    [first, ..rest] -> ___________________________________
  }
}
```

2 Algebraic Data Types

An algebraic data type is a type built from other types in two ways: by combining them, called a product type, or by choosing between them, called a sum type. We can implement a product type using a tuple. `#(Int, Bool, String)` contains an Int, a Bool, and a String. Gleam allows defining sum types by specifying variants of a type. Each variant has its own constructor. We instantiate sum types by calling a constructor of a variant.
A `LinkedList(a)` is either a `Link` holding a value and another list, or `Empty`. Each variant is itself a product of its fields. To use a sum type, you pattern match with `case`.

```
pub type LinkedList(a) {
  Link(first: a, rest: LinkedList(a))
  Empty
}
```

1. Implement `extend`, which takes in two LinkedLists `a` and `b` and returns a LinkedList with the elements of `b` after the elements of `a`.

```
pub fn extend(a: LinkedList(t), b: LinkedList(t)) -> LinkedList(t) {
  case a {
    Empty -> ____________
    Link(first, rest) -> ________________________________________
  }
}
```

2. Implement `split`, which takes in a LinkedList `l` and a value `at`, and returns a tuple of the parts of `l` before and after the first occurrence of `at`. `at` is the first element of the second part. If `at` is not found, return `#(l, Empty)`.

```
pub fn split(l: LinkedList(a), at: a) -> #(LinkedList(a), LinkedList(a)) {
  case l {
    Empty -> #(Empty, Empty)
    Link(first, _) if first == at -> ______________________
    Link(first, rest) -> {
      let #(before, after) = ______________________
      ____________________________________
    }
  }
}
```

3. Implement `invert`, which takes in a LinkedList `list` and a value `at`, and moves the elements starting from the first occurrence of `at` to the front, before the elements that came before `at`. For example, inverting `1 -> 2 -> 3 -> 4` at `3` gives `3 -> 4 -> 1 -> 2`. If `at` is not found, return `list` unchanged. You may use `split` and `extend`.

```
pub fn invert(list: LinkedList(a), at: a) -> LinkedList(a) {
  let #(before, after) = ______________________
  ______________________
}
```

Just like in Python, a `Tree` in Gleam contains a `label` and a list of `Tree`s as `branches`. `label` can be of a generic type, called `a` in our implementation. `leaf` returns a `Tree` with label `label` and an empty list for `branches`.

```
pub type Tree(a) {
    Tree(label: a,
        branches: List(Tree(a)))
}

pub fn leaf(label: a) -> Tree(a) {
    Tree(label, [])
}
```

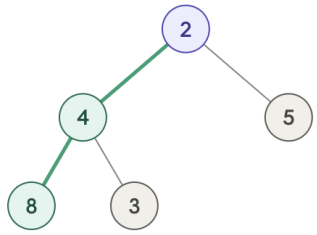
4. Implement `prune_even`, which prunes every branch in a tree that has an even label. Odd child branches of branches with even labels are not preserved.

```
pub fn prune_even(tree: Tree(Int)) -> Tree(Int) {
    let Tree(label, branches) = tree

    let remove_even = list.filter(branches, ____________________________)
    let prune_children = list._________(___________________________________)

    Tree(label, prune_children)
}
```

5. A *treemil* is a tree in which at least one of the child node(s) of each non-leaf node has a label that is a multiple of the label of the node. Alternatively, a treemil is a tree where every node has at least one path from the node to a leaf consisting entirely of nodes that are multiples of its parent. Implement `is_treemil`, which returns whether a tree is a treemil.




```
pub fn is_treemil(tree: Tree(Int)) -> Bool {
    // valid if:
    // 1. at least one child node is a multiple of its parent
    // 2. all children are valid treemils
    // leaves are valid treemils

    case tree {
        //is a leaf
        Tree(_, []) -> _________

        //is not a leaf
        Tree(label, branches) -> {
            let child_multiples = list._______(_______, _________________)
            case list._________(child_multiples) >= 1 {
                True -> list._________(____________________________________)
                False -> _________
            }
        }
    }
}
```

6. Implement `max_treemil` which takes in a tree `t` and determines the highest number of nodes in a treemil that is a subtree of `t`. You may assume `is_treemil` is implemented correctly from the previous question.

```
pub fn max_treemil(tree: Tree(Int)) -> Int {
    //Given a tree (that is not necessarily a treemil), returns
    //the highest number of nodes in a treemil in the tree.

    case tree {
        //is a leaf
        Tree(_, []) -> _________

        //is not a leaf
        Tree(_, branches) -> {
            case ___________________________________ {
                True -> 1 + int._________(___________________________________)
                False -> result.unwrap(list.max(_________, int.compare), 0)
            }
        }
    }
}
```